AI Audit Report

Student ID: 23127081
Full Name: Nguyễn Phan Hùng Linh

Declaration

I use AI tools for the following tasks. I used a second Codex/GPT-5.6 agent as an assistant. I reviewed its suggestions and raw-log calculations myself. The agent did not run the final measurements, create the JTL files, or make the final decisions.

Interaction 1 – Test design consultation

Field	Record
AI tool	Codex / GPT-5.6
Date and time	2026-08-16 ICT
Student prompt	“Read HW05.md and inspect the eshop-sut project read-only. Recommend a simple, compliant plan: deliverables, non-conflicting endpoints for Load/Stress/Spike, test scenarios/metrics, likely tools. Give concise advice and leave an audit-ready account of your consultation. Do not edit files.”

AI output

The AI suggested this initial mapping: Load `GET /api/orders/{order_id}` with `data/order_ids.csv`; Stress `POST /api/register` with `data/register_users.csv`; Spike `POST /api/apply-coupon` with `data/coupons.csv`. It suggested 20 virtual users, a 60-second ramp, and a five-minute steady period for Load; a 10-to-50 user stress run; and a 2 → 40 → 2 user spike. It proposed Summary Report, Aggregate Report, and View Results Tree as different listeners. It also warned that View Results Tree can use much memory, asked for HTTP/JSON assertions, and noted that `GET /api/orders/{id}` is public and this should be reported as an observation.

My review and correction

The advice was a useful starting point, but it missed my instructor's special requirement to choose **three endpoints for every scenario**. I changed the scope to three-endpoint workflows, added three separate CSV files, and used real smoke tests before the measured runs. I also used 30, not 50, stress users for the first real write-heavy test.

Interaction 2 – Raw-result analysis consultation

Field	Record
AI tool	Codex / GPT-5.6
Date and time	2026-08-16 ICT
Student prompt	“Read the raw JTL files in results/load.jtl, results/stress.jtl, results/spike.jtl, and results/endurance.jtl plus their resource CSVs. Analyze performance, propose thresholds and optimizations. Be concise but show numbers and conclusions. Do not edit files. I will independently check all values.”

AI output

The AI reported these estimates:

Run	Samples	RPS	Error rate	Mean	p95	p99	Maximum
Load	5,348	17.90	0.00%	8.56 ms	30 ms	68 ms	178 ms
Stress	8,522	47.55	0.00%	25.06 ms	139 ms	339 ms	648 ms

| Spike | 3,711 | 18.77 | 0.00% | 22.05 ms | 97 ms | 403 ms | 622 ms |
| Endurance | 55,622 | 92.89 | 0.00% | 19.04 ms | 83 ms | 293 ms | 1,031 ms |

It reported backend CPU/RSS average and peak values as Load 1.99%/17.3% and 33,530/45,744 KB; Stress 9.17%/24.6% and 42,382/64,304 KB; Spike 3.72%/18.0% and 34,357/63,664 KB; Endurance 7.76%/26.9% and 57,294/76,000 KB.

The AI suggested CI gates of error rate at most 1%, endurance p95 at most 150 ms, p99 at most 600 ms, and throughput at least 85 RPS. It classified an index on `orders(user_id, id)`, an index on `users(email)`, and SQLite WAL as feasible experiments. It classified a normal connection pool as unsuitable for this local SQLite SUT and said caching is not justified by the current evidence. It also stated that 92.89 RPS is a tested stable level, not a proven maximum hardware capacity.

My review and correction

I recalculated values from the JTL elapsed column. The Load, Stress, and Endurance percentile values agreed. The AI's merged Spike p95 was not exact: it wrote 97 ms, but the sorted `results/spike.jtl` value is **93 ms**. It also wrote 105 ms for the spike-stage p95; the stage JTL value is **103 ms**. I use the direct raw values in the main report. I use 92.70 RPS in the report because I divide by the planned 600 seconds; the AI divided by its observed elapsed window. This is a denominator difference, so neither value proves a capacity limit.

Human responsibility

I verified that the resource-monitor PID was `/opt/homebrew/bin/node server.js`. I kept the raw files and reported only measured performance values.